

形式语言与编译第四次作业

计算机 2303 尤颢辰

2026 年 5 月 19 日

1 习题 7.1

1.1 $w = 010010$

下面列出成功路径:

$$\begin{aligned}(q_0, 010010, Z_0) &\vdash (q_2, 010010, Z_0) \\ &\vdash (q_3, 10010, 0Z_0) \\ &\vdash (q_4, 0010, 0Z_0) \\ &\vdash (q_4, 010, 0Z_0) \\ &\vdash (q_4, 10, 0Z_0) \\ &\vdash (q_5, 0, 0Z_0) \\ &\vdash (q_6, \varepsilon, Z_0) \\ &\vdash (q_8, \varepsilon, Z_0)\end{aligned}$$

1.2 $w = 100101001$

下面列出成功路径:

$$\begin{aligned}(q_0, 100101001, Z_0) &\vdash (q_1, 100101001, Z_0) \\ &\vdash (q_2, 00101001, Z_0) \\ &\vdash (q_3, 0101001, 0Z_0) \\ &\vdash (q_3, 101001, 00Z_0) \\ &\vdash (q_4, 01001, 00Z_0) \\ &\vdash (q_4, 1001, 00Z_0) \\ &\vdash (q_5, 001, 00Z_0) \\ &\vdash (q_6, 01, 0Z_0) \\ &\vdash (q_6, 1, Z_0) \\ &\vdash (q_7, \varepsilon, Z_0) \\ &\vdash (q_8, \varepsilon, Z_0)\end{aligned}$$

2 习题 7.2(2)

设计所有 0 的个数是 1 的个数 2 倍的 0-1 串的集合:

题目要求设计的语言是：

$$L = \{w \in \{0,1\}^* \mid \text{num}_0(w) = 2\text{num}_1(w)\}.$$

核心思路：让 PDA 用栈记录 $\text{num}_0(w) - 2\text{num}_1(w)$ 。

构建 PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

其中：

$Q = \{q_0, p, q_f\}$ ，分别为起始状态，中间状态，接受状态。

$\Sigma = \{0, 1\}$

$\Gamma = \{A, B, Z_0\}$ 若当前 0 的数量较多，则用栈符号 A 表示多出的 0；若当前 0 的数量不足，则用栈符号 B 表示还缺少的 0。其中每个 A 表示多出一个 0，每个 B 表示还缺少一个 0。

当前状态	输入符号	栈顶符号	下一状态	栈操作说明
q_0	0	Z_0	q_0	$Z_0 \mapsto AZ_0$
q_0	0	A	q_0	$A \mapsto AA$
q_0	0	B	q_0	$B \mapsto \varepsilon$
q_0	1	Z_0	q_0	$Z_0 \mapsto BBZ_0$
q_0	1	B	q_0	$B \mapsto BBB$
q_0	1	A	p	$A \mapsto \varepsilon$
p	ε	A	q_0	$A \mapsto \varepsilon$
p	ε	Z_0	q_0	$Z_0 \mapsto BZ_0$
q_0	ε	Z_0	q_f	$Z_0 \mapsto Z_0$

上表列出了 PDA 的转移函数，这样，我们就构建了题目要求的 PDA。

3 习题 7.7(1)

例 7.3 的 PDA $L = \{w \mid w = w^R, w \in \{0,1\}^*\}$ 不是 DPDA。

判断一个 PDA 是否确定，可以看在同一个瞬时状态下，是否会有多个可能动作。

我们发现，对这个 PDA，有：

$$\delta(q_0, 0, \varepsilon) = \{(q_0, 0), (q_1, \varepsilon)\}$$

因此， $|\delta(q_0, 0, \varepsilon)| = 2 > 1$ ，所以其不是 DPDA。

4 习题 8.1(2)

$$P \rightarrow \hat{D}\hat{S}$$

$$\hat{D} \rightarrow \varepsilon \mid \hat{D}D;$$

$$D \rightarrow \text{int } d$$

$$\hat{S} \rightarrow \varepsilon \mid S; \hat{S}$$

$$S \rightarrow d = E$$

$$E \rightarrow d \mid E + i$$

对应的推导为：

$$\begin{aligned}
 \hat{D}D; d = E; \hat{S} &\Rightarrow D; d = E; \hat{S} \\
 &\Rightarrow \text{int } d; d = E; \hat{S} \\
 &\Rightarrow \text{int } d; d = E + i; \hat{S} \\
 &\Rightarrow \text{int } d; d = d + i; \hat{S} \\
 &\Rightarrow \text{int } d; d = d + i;
 \end{aligned}$$

令 $d = x, x = 1$ ，则有 $\text{int } x; x = x + 1;$

所以符号串 $\hat{D}D; d = E; \hat{S}$ 是 $\text{int } x; x = x + 1;$ 的一个句型。

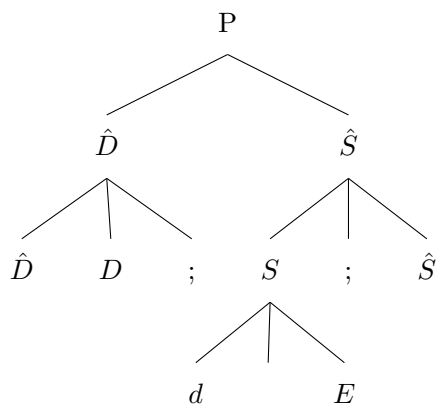
下面尝试用最右推导来得到这个巨型。如果其为规范句型应该能得到。最右推导：

$$\begin{aligned}
 P &\Rightarrow \hat{D}\hat{S} \\
 &\Rightarrow \hat{D}S; \hat{S} \\
 &\Rightarrow \hat{D}S; \\
 &\Rightarrow \hat{D}d = E; \\
 &\Rightarrow \hat{D}d = E + i; \\
 &\Rightarrow \hat{D}d = d + i;
 \end{aligned}$$

可以看到，在最右推导中，只要先展开 $\hat{S}$ ，就会得到赋值语句部分；而题目中的句型 $\hat{D}D; d = E; \hat{S}$ 同时保留了右侧的 $\hat{S}$ ，又已经把它左边的一部分展开成了 $d = E$ 。这说明它不是按照最右推导顺序得到的。

所以我们得出结论， $\hat{D}D; d = E; \hat{S}$ 不是规范句型。

下面我们构造语法树找短语：



每个已展开子树的边界（叶子串）就是一个短语。所有的短语为：

$$\begin{aligned}
 &\hat{D}D; d = E; \hat{S} \\
 &\hat{D}D; \\
 &d = E; \hat{S}
 \end{aligned}$$

$$d = E$$

其中，直接短语有：

$$\hat{D}D;$$

$$d = E$$

至于句柄，理论上最短的直接短语就是句柄。但是由于这个句型不是规范句型，所以严格来讲没有句柄。

5 习题 8.2(1)

题目文法为：

$$S \rightarrow a \mid \Lambda \mid (T) \quad T \rightarrow T, S \mid S$$

目标句子为：

$$(((a, a), \Lambda, (a)), a)$$

要求其规范推导，可以列出文法的最右推导如下：

$S \Rightarrow (T)$	句柄: (T)
$\Rightarrow (T, S)$	句柄: T, S
$\Rightarrow (T, a)$	句柄: a
$\Rightarrow (S, a)$	句柄: S
$\Rightarrow ((T), a)$	句柄: (T)
$\Rightarrow ((T, S), a)$	句柄: T, S
$\Rightarrow ((T, (T)), a)$	句柄: (T)
$\Rightarrow ((T, (S)), a)$	句柄: S
$\Rightarrow ((T, (a)), a)$	句柄: a
$\Rightarrow ((T, S, (a)), a)$	句柄: T, S
$\Rightarrow ((T, \Lambda, (a)), a)$	句柄: Λ
$\Rightarrow ((S, \Lambda, (a)), a)$	句柄: S
$\Rightarrow (((T), \Lambda, (a)), a)$	句柄: (T)
$\Rightarrow (((T, S), \Lambda, (a)), a)$	句柄: T, S
$\Rightarrow (((T, a), \Lambda, (a)), a)$	句柄: a
$\Rightarrow (((S, a), \Lambda, (a)), a)$	句柄: S
$\Rightarrow (((a, a), \Lambda, (a)), a)$	句柄: a

6 习题 8.3

下面证明文法：

$$\begin{aligned} S &\rightarrow A \\ A &\rightarrow Ab \mid bBa \\ B &\rightarrow aAc \mid a \mid aAb \end{aligned}$$

不是 LR(0) 且是 SLR(1) 文法。

首先，我们证明该文法不是 LR(0)：找出 LR(0) 项目集中的冲突。

增广文法为：

$$S' \rightarrow S$$

先构造部分 LR(0) 项目集。初态为：

$$I_0 = \begin{cases} S' \rightarrow \cdot S \\ S \rightarrow \cdot A \\ A \rightarrow \cdot Ab \\ A \rightarrow \cdot bBa \end{cases}$$

由 I_0 经 A 转移得到：

$$I_1 = \begin{cases} S \rightarrow A \cdot \\ A \rightarrow A \cdot b \end{cases}$$

这里产生了移进-规约冲突。项目 $S \rightarrow A \cdot$ 表示可以按 $S \rightarrow A \cdot$ 规约，项目 $A \rightarrow A \cdot b$ 表示遇到 b 时可以移进。所以该文法不是 LR(0) 文法。

其次，我们证明该文法是 SLR(1) 文法。

首先关于项目集 I_1 ，它在 LR(0) 中有移进-规约冲突。在 SLR(1) 中， $S \rightarrow A$ 只在 $\text{FOLLOW}(S) = \{\$ \}$ 上规约，而移进只发生在 b 上，所以不冲突。

其次关于 $I_2 = \{A \rightarrow Ab \cdot, B \rightarrow aAb \cdot\}$ ，它在它在 LR(0) 中有归约-归约冲突。在 SLR(1) 中： $A \rightarrow Ab$ 只在 $\text{FOLLOW}(A) = \{\$, b, c\}$ 上归约； $B \rightarrow aAb$ 只在 $\text{FOLLOW}(B) = \{a\}$ 上归约。因为：

$$\text{FOLLOW}(A) \cap \text{FOLLOW}(B) =$$

所以不冲突。

除上述两个状态外，其余项目集均无冲突。所以，该文法是 SLR(1) 文法。

7 习题 9.1(1)

文法为：

$$G_e: \quad S \rightarrow \varepsilon \mid (S) \mid SS$$

设 $S.l$ 表示由 S 推导出的括号串的最大嵌套深度

语义规则如下：

产生式	语义规则
$S \rightarrow \varepsilon$	$S.l = 0$
$S \rightarrow (S_1)$	$S.l = S_1.l + 1$
$S \rightarrow S_1 S_2$	$S.l = \max(S_1.l, S_2.l)$

即 $S.l$ 便为我们要求的 $\text{att}(G(e))$ 。

8 习题 9.1(2)

设属性含义为：

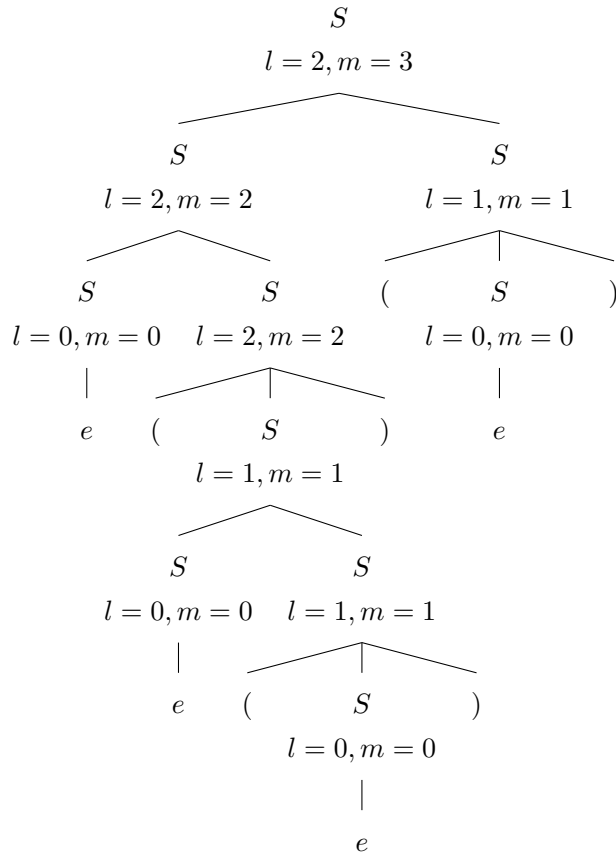
$S.l$ 表示由 S 推导出的括号串的最大嵌套深度；

$S.m$ 表示由 S 推导出的句子中的括号对数。

文法和语义规则为：

产生式	语义规则
$S \rightarrow e$	$S.l = 0, \quad S.m = 0$
$S \rightarrow (S_1)$	$S.l = S_1.l + 1, \quad S.m = S_1.m + 1$
$S \rightarrow S_1 S_2$	$S.l = \max(S_1.l, S_2.l), \quad S.m = S_1.m + S_2.m$

列出其带注释语法树：



下面列出该句子的语法制导的属性求值过程：

约定：

$$\begin{aligned} r_1: S &\rightarrow e \\ r_2: S &\rightarrow (S) \\ r_3: S &\rightarrow S_1 S_2 \end{aligned}$$

则有语法制导的属性求值过程：

符号栈	<i>l</i> 栈	<i>m</i> 栈	剩余串	动作
#	#	#	<i>e</i> (<i>e</i>)(<i>e</i>)#	移进
# <i>e</i>	#-	#-	(<i>e</i>)(<i>e</i>)(<i>e</i>)#	$r_1: S.l = 0, S.m = 0$
# <i>S</i>	#0	#0	(<i>e</i>)(<i>e</i>)(<i>e</i>)#	移进
# <i>S</i> (	#0-	#0-	<i>e</i> (<i>e</i>)(<i>e</i>)#	移进
# <i>S</i> (<i>e</i>	#0--	#0--	(<i>e</i>)(<i>e</i>)#	$r_1: S.l = 0, S.m = 0$
# <i>S</i> (<i>S</i>	#0-0	#0-0	(<i>e</i>)(<i>e</i>)#	移进
# <i>S</i> (<i>S</i> (	#0-0-	#0-0-	<i>e</i>)(<i>e</i>)#	移进
# <i>S</i> (<i>S</i> (<i>e</i>	#0-0--	#0-0--	))(<i>e</i>)#	$r_1: S.l = 0, S.m = 0$
# <i>S</i> (<i>S</i> (<i>S</i>	#0-0-0	#0-0-0	))(<i>e</i>)#	移进
# <i>S</i> (<i>S</i> (<i>S</i> (	#0-0-0-	#0-0-0-	)(<i>e</i>)#	$r_2: S.l = 0 + 1 = 1, S.m = 0 + 1 = 1$
# <i>S</i> (<i>SS</i>	#0-01	#0-01	)(<i>e</i>)#	$r_3: S.l = \max(0, 1) = 1, S.m = 0 + 1 = 1$
# <i>S</i> (<i>S</i>	#0-1	#0-1	)(<i>e</i>)#	移进
# <i>S</i> (<i>S</i>)	#0-1-	#0-1-	(<i>e</i>)#	$r_2: S.l = 1 + 1 = 2, S.m = 1 + 1 = 2$
# <i>SS</i>	#02	#02	(<i>e</i>)#	$r_3: S.l = \max(0, 2) = 2, S.m = 0 + 2 = 2$
# <i>S</i>	#2	#2	(<i>e</i>)#	移进
# <i>S</i> (	#2-	#2-	<i>e</i>)#	移进
# <i>S</i> (<i>e</i>	#2--	#2--	)#	$r_1: S.l = 0, S.m = 0$
# <i>S</i> (<i>S</i>	#2-0	#2-0	)#	移进
# <i>S</i> (<i>S</i>)	#2-0-	#2-0-	#	$r_2: S.l = 0 + 1 = 1, S.m = 0 + 1 = 1$
# <i>SS</i>	#21	#21	#	$r_3: S.l = \max(2, 1) = 2, S.m = 2 + 1 = 3$
# <i>S</i>	#2	#3	#	acc

9 习题 9.3(1)

根据课程资料，假设：

$$\text{sizeof}(\text{INT}) = \text{sizeof}(\text{FLO}) = 4, \text{sizeof}(\text{FUNC}) = \text{sizeof}(\text{FUNPTT}) = 8, \text{sizeof}(\text{LARR}) = 4.$$

这一小问的声明为：

```
int x; int a[5]; float b[3,6];
```

翻译后的符号表为：

```
1 @table: (
2   outer: NIL
```

```

3 width: 96
4 argc: 0
5 arglist: NIL
6 rtype: VOID
7 entry: (name: x type: INT offset: 4)
8 entry: (name: a type: ARRAY base: 24 etype: INT dims: 1 dim[0]: 5)
9 entry: (name: b type: ARRAY base: 96 etype: FLO dims: 2 dim[0]: 3 dim[1]: 6)
10 )

```

10 习题 9.3(4)

假设同 9.3(1);

这一小问的声明为:

```
int h(int f(nil);int y;)int g(int c[];)S;S;
```

定义三个符号表,@table,h@table,g@table;

其翻译后的符号表为:

```

1 @table: (
2   outer: NIL
3   width: 8
4   argc: 0
5   arglist: NIL
6   rtype: VOID
7   entry: (name: h type: FUNC offset: 8 mytab: h@table)
8 )
9 h@table: (
10  outer: @table
11  width: 20
12  argc: 2
13  arglist: (f y)
14  rtype: INT
15  entry: (name: f type: FUNPTT offset: 8 rtype: INT)
16  entry: (name: y type: INT offset: 12)
17  entry: (name: g type: FUNC offset: 20 mytab: g@table)
18 )
19 g@table: (
20  outer: h@table
21  width: 4
22  argc: 1
23  arglist: (c)
24  rtype: INT
25  entry: (name: c type: LARR offset: 4 etype: INT)
26 )

```

最终结构关系是:

$$@table \longrightarrow h@table \longrightarrow g@table$$

其中 h 登记在当前符号表中, f,y,g 登记在 h@table 中, c 登记在 g@table 中。